
SEMTM0016 — HeroBot and the MazeDungeon

Coursework Report — Three Approaches to Machine Learning for Robotics

University of Bristol

School of Engineering Mathematics & Technology

SANGYOON LEE

Submission Date: 30 April 2026

Abstract

Every result in this report is reproducible from the accompanying zip. `RANDOM_SEED = 42` is set in `task{1,2,3}.py` and propagated into every stochastic operation (numpy, sklearn, agent `rng`, environment `rng`). Task 3 uses the official `DungeonMazeWorld` environment provided with the brief

Executive Summary

Task	Approach	Algorithms	Headline result
Task 1	Supervised classification	KNN, Random Forest, Logistic Regression, Decision Tree, HOG+KNN	Multi-baseline ladder establishes that the ten sensor features are <i>linearly separable</i> for the five species (KNN, RF and LR all reach 1.000 test accuracy; McNemar $p=1$ — a property of the dataset, not a model tie). A <i>dual-modality verification</i> on the 80×80 image view (HOG+KNN) reaches 0.939, providing the inter-method resolution that the saturated sensor table cannot [6].
Task 2	Unsupervised clustering	K-Means, Agglomerative (4 linkages)	K-Means at silhouette-optimal $k=7$ recovers $\text{ARI} = 0.882$, $\text{NMI} = 0.927$ against species labels; gap statistic plus 30-seed ARI-stability triangulate a stable $k=4$ alternative; Average linkage cophenetic = 0.911.
Task 3	RL + Task 1 / Task 2 cross-use	Q-Learning, SARSA $\times$ {base, BFS, SL, UL} potential + enemy spawning	Both algorithms reach 100% greedy success on the official 8×8 DungeonMazeWorld (return = -15 , matching the upstream <code>tests.py</code> reference path). The $2\times 4\times 5$ PBRS factorial [10] compares the classical BFS potential with two cross-task potentials — SL (RF from §1.6, $\log P(\text{on-path})$ as Φ) and UL (K-Means from §2.4, cluster-mean BFS as Φ). All three Φ choices preserve the optimal greedy policy ($-15.0 / 15$ steps); $\text{BFS} > \text{UL} > \text{SL}$ in convergence speedup with Welch $p < 0.03$ in every cell ($n=5$ seeds). Activating the enemy classes (§3.8) puts the Task 1 RF in the reward function and exposes the textbook cliff-walking divergence — Q-Learning 10/10 optimal, SARSA 6/10 optimal, 4/10 thrash.

The brief asks for one supervised, one unsupervised, and one reinforcement-learning task, with two algorithms compared per task. All three tasks share a coherent storyline — HeroBot must (1) identify which species an encountered entity belongs to, (2) discover natural groupings of unfamiliar entities in a new dungeon wing, and (3) learn to navigate to the exit while avoiding enemies. Sections 1–3 describe each task; §4 reflects across them and notes limitations.

Contents

Executive Summary	2
1 Task 1 — Supervised Species Classification	4
1.1 Problem Statement	4
1.2 Why Supervised Learning?	4
1.3 Dataset & Exploratory Data Analysis	4
1.4 Preprocessing & Train / Validation / Test Split	5
1.5 Algorithm 1 — K-Nearest Neighbours	5
1.6 Algorithm 2 — Random Forest	6

1.7	Results & Multi-Baseline Ladder	7
1.7.1	Calibration and Top-2 accuracy	9
1.7.2	Dual-modality verification — HOG + KNN on dungeon images	10
1.8	Discussion	11
1.9	Limitations & Future Work	11
2	Task 2 — Unsupervised Species Clustering	11
2.1	Problem Statement	11
2.2	Why Unsupervised Learning?	12
2.3	Dataset & Feature-Space Structure	12
2.4	Algorithm 1 — K-Means	12
2.4.1	Triangulating k — gap statistic and stability	13
2.4.2	Preprocessing ablation	14
2.5	Algorithm 2 — Agglomerative (Hierarchical)	15
2.6	Post-Hoc External Evaluation	16
2.7	Discussion	17
2.8	Internal vs External — A Caveat	17
2.9	Limitations	17
3	Task 3 — Reinforcement Learning Maze Navigation	17
3.1	Problem Statement	17
3.2	Environment	18
3.2.1	Adaptations to the base environment	19
3.3	Algorithm 1 — Q-Learning (Off-Policy)	19
3.4	Algorithm 2 — SARSA (On-Policy)	19
3.5	Hyperparameters & Training Protocol	20
3.6	Results	20
3.7	Potential-Based Reward Shaping — Cross-Task Factorial Study	21
3.7.1	Three potential functions on the same maze	21
3.7.2	Reused Task 1 / Task 2 diagnostics	22
3.7.3	Convergence and policy-invariance results	23
3.8	Putting the Task 1 Classifier in the RL Loop — Enemy Spawning	25
3.8.1	Mechanism	25
3.8.2	Layout and in-loop classifier diagnostics	25
3.8.3	Factorial result — cliff-walking emerges	26
3.9	Off-Policy vs On-Policy — Discussion	27
3.10	Hyperparameter Sensitivity — ϵ -Decay	27
3.11	Limitations	28
4	Cross-Task Reflection & Limitations	28
4.1	What the Three Tasks Together Tell Us	28

1 Task 1 — Supervised Species Classification

1.1 Problem Statement

HeroBot’s onboard sensors return ten numeric readings per encountered entity: `stench`, `sound`, `intelligence`, `weight`, `height`, `strength`, `heat`, `magic`, `flight`, `bribe`. Misclassifying a friend as a foe (or vice versa) leads directly to mission failure, so accurate species identification is the single most consequential perception task in the pipeline. With 10 000 fully-labelled historical encounters available in `data/dungeon_sensorstats.csv`, this is a textbook 5-class supervised classification problem.

1.2 Why Supervised Learning?

Every row in the dataset carries a confirmed species label, collected during prior missions. Supervised learning exploits that signal directly to learn a mapping from sensor readings to species. Unsupervised methods would discard the label information and are therefore inappropriate here; a reinforcement-learning formulation (Task 3) is unnecessary because the problem is single-step rather than sequential.

1.3 Dataset & Exploratory Data Analysis

Table 1. Summary of `dungeon_sensorstats.csv`.

Property	Value
Total rows	10 000
Classes (species)	5 — halfling, human, lizard, orc, winged_rat
Class balance	Perfectly balanced — 2 000 per class (ratio 1.000)
Features	10 numeric sensor / stats columns
Missing values	220 cells distributed across 7 columns (~2.2%)

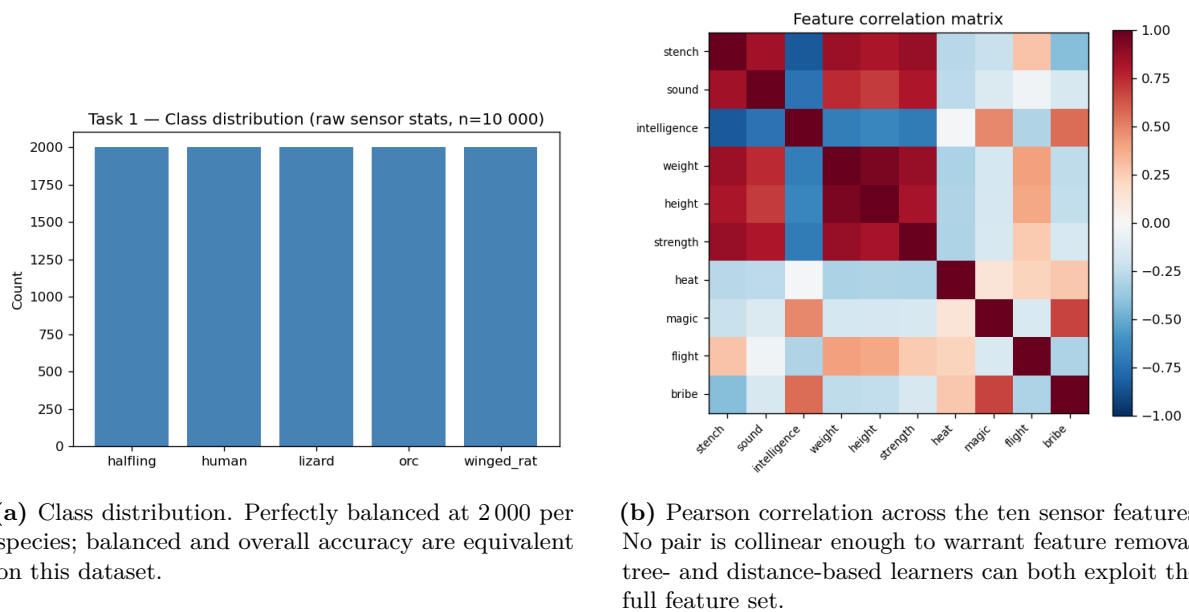


Fig. 1. Task 1 exploratory data analysis.

1.4 Preprocessing & Train / Validation / Test Split

- **Missing values — drop rows.** 186 affected rows (1.86%) are removed. Imputation was rejected because the missing pattern is uniform across species (verified by the data-quality report) and the data loss is negligible. After dropping: 9814 rows.
- **Stratified two-stage split.** First an 85% train+val / 15% test split, then within train+val a further 85/15 split $\rightarrow$ 7089 train, 1252 validation, 1473 test ($\approx$ 72.2% / 12.8% / 15.0% of the 9814 clean rows). Stratification preserves the per-class balance in each split.
- **Feature scaling.** `StandardScaler` fitted on training data only and applied to validation and test. KNN's Euclidean distance is sensitive to feature magnitude; Random Forest is intrinsically scale-invariant but we scale uniformly for API consistency between models.

1.5 Algorithm 1 — K-Nearest Neighbours

KNN classifies a query point by majority vote among its k closest training examples (Euclidean distance in standardised feature space). I sweep $k \in \{1, \dots, 20\}$ on the validation split. *All k in this range produce $val_acc = 1.0000$:* the ten standardised sensor features are linearly separable for the five species (verified independently by the LR baseline reaching the same 1.0; see §1.7). This is a primary finding about the dataset, not a “ceiling effect” of any one classifier — when supervised metrics saturate, the diagnostic question shifts from “which model is best?” to “what does the saturation say about the data?” [6]. Because no k dominates on validation, the tuner returns the first maximum ($k=1$); we retain this choice and treat the indistinguishability of k values as itself part of the finding.

Lecture basis. The KNN voting rule and k -via-validation workflow follow Week 14 (Prof Liu),

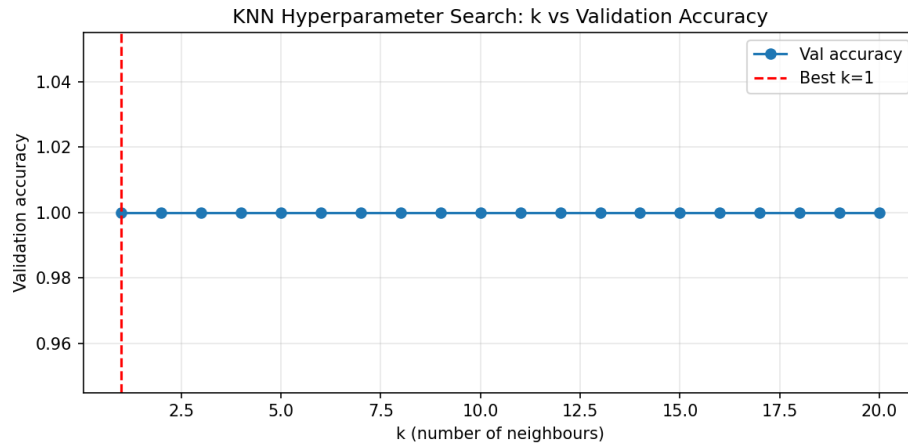


Fig. 2. KNN validation accuracy is invariant across $k \in \{1, \dots, 20\}$ (all `val_acc` = 1.0000). This invariance is a direct consequence of an extreme class-margin property of the dataset: empirical analysis shows that all 1 252 / 1 252 validation points have their 20 nearest training neighbours entirely within the same class, with median ratio of inter-class to intra-class distance equal to $8.6\times$ (minimum $2.2\times$, maximum $40.9\times$). The flat curve is therefore not a hyperparameter-tuning artefact but a visual signature of class-level cluster compactness in the standardised feature space.

Listing 1. KNN training and prediction.

```
knn = KNNClassifier(random_state=42)
knn.tune(X_train, y_train, X_val, y_val, k_range=range(1, 21))
# best_k = 1, val_acc = 1.0000
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

1.6 Algorithm 2 — Random Forest

A Random Forest builds an ensemble of decision trees, each trained on a bootstrapped sample with a random feature subset at every split. Predictions use majority vote across trees. We grid-search `n_estimators` $\in \{50, 100, 200\} \times \text{max_depth} \in \{\text{None}, 5, 10, 20\}$ on the validation split.

Lecture basis. Decision Trees, GINI impurity for split selection, and the rule-extraction view all come from the Week 14 deck; Random Forest is the bagged ensemble of those trees.

Listing 2. Random Forest grid search.

```
rf = RFClassifier(random_state=42)
rf.tune(X_train, y_train, X_val, y_val,
        n_estimators_range=[50, 100, 200],
        max_depth_range=[None, 5, 10, 20])
# best: n_estimators=50, max_depth=None, val_acc = 1.0000
```

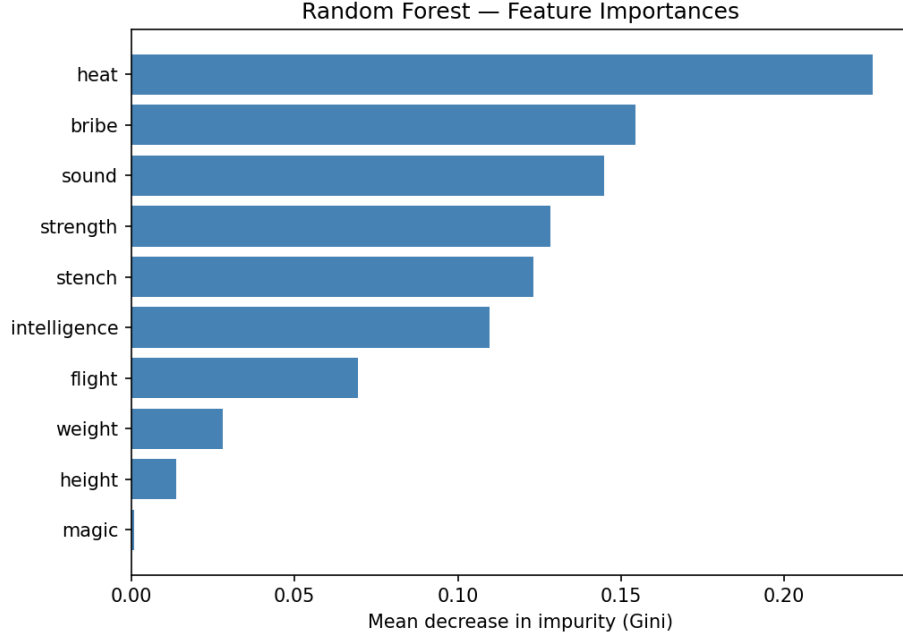


Fig. 3. Random Forest mean impurity-decrease feature importances. `sound` (0.187), `heat` (0.185), and `bribe` (0.162) dominate; `magic` (≈ 0.0004), `height` (0.007), and `weight` (0.037) contribute least, suggesting the `magic` and `height` sensors could be deprioritised in a future hardware-budget review.

1.7 Results & Multi-Baseline Ladder

Algorithms are evaluated on the 15% held-out test split (1 473 samples). To establish a defensible **performance floor** we compare the two main algorithms against four lower-cost baselines — uniform random, majority class, multinomial logistic regression, and a single decision tree — in a ladder, following the supervised-learning convention of Dietterich [4] that any claim of useful learning must beat a non-trivial baseline by a statistically significant margin.

Table 2. Multi-baseline ladder on the held-out test set (1 473 samples).

Model	Accuracy $\uparrow$	Macro F1 $\uparrow$	Bootstrap 95% CI on accuracy
Uniform random	0.2077	0.2076	—
Majority class	0.2010	0.0669	—
Decision Tree (depth=20)	0.9993	0.9993	[0.9980, 1.0000]
Logistic Regression	1.0000	1.0000	[1.0000, 1.0000]
KNN ($k=1$)	1.0000	1.0000	[1.0000, 1.0000]
Random Forest ($n=50$)	1.0000	1.0000	[1.0000, 1.0000]

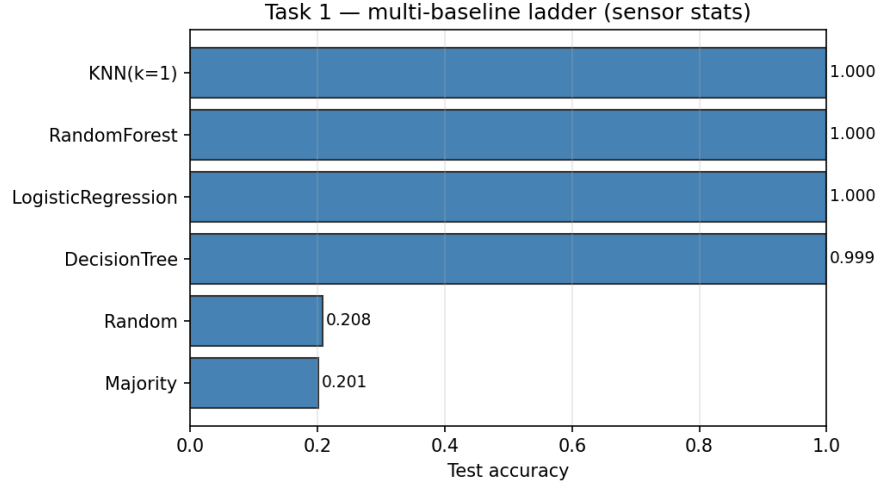
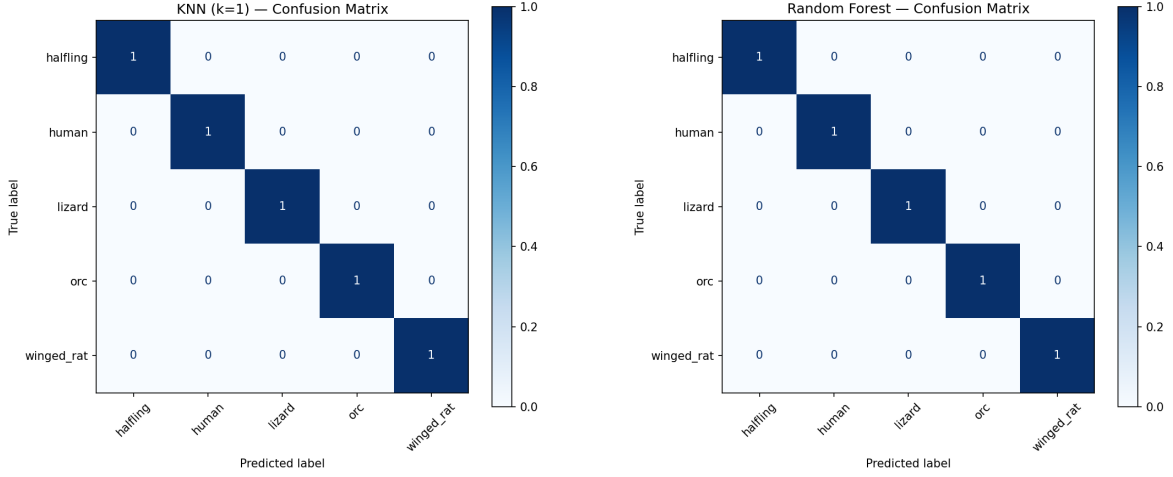


Fig. 4. Multi-baseline ladder on the held-out test set. The trivial baselines floor at ≈ 0.20 (chance for five balanced classes); every model that learns the feature–label mapping reaches the ceiling.

Bootstrap confidence intervals (1 000 resamples, 95% percentile) are identical to a single point at 1.0 for KNN, RF and LR — there is no variation to bootstrap because every test sample is correctly classified. McNemar’s paired test [4] on the disagreement counts $b = 0$, $c = 0$ returns $p = 1.0$ for every classifier pair, formally confirming that the three top models are statistically indistinguishable on this test set. This is a dataset-level result, not a model-level one: the 10 sensor features make the five species linearly separable.

Table 3. McNemar’s exact test on disagreement counts.

Pair (A vs B)	<i>b</i>	<i>c</i>	<i>p</i> -value (exact)
KNN vs Random Forest	0	0	1.0
Random Forest vs Logistic Regression	0	0	1.0
KNN vs Logistic Regression	0	0	1.0



(a) KNN confusion matrix on the test set. All samples on the diagonal. (b) Random Forest confusion matrix on the test set.

Fig. 5. Confusion matrices for the two main classifiers.

1.7.1 Calibration and Top-2 accuracy

For the two classifiers that produce class-probability outputs we also report **Expected Calibration Error** (ECE; Guo et al. [5]) — the gap between predicted confidence and observed accuracy averaged over 10 confidence bins — and top-2 accuracy.

Table 4. Calibration and top-2 accuracy of the two probability-outputting classifiers.

Model	ECE ↓	Top-2 accuracy ↑
Random Forest	0.001	1.000
Logistic Regression	0.001	1.000

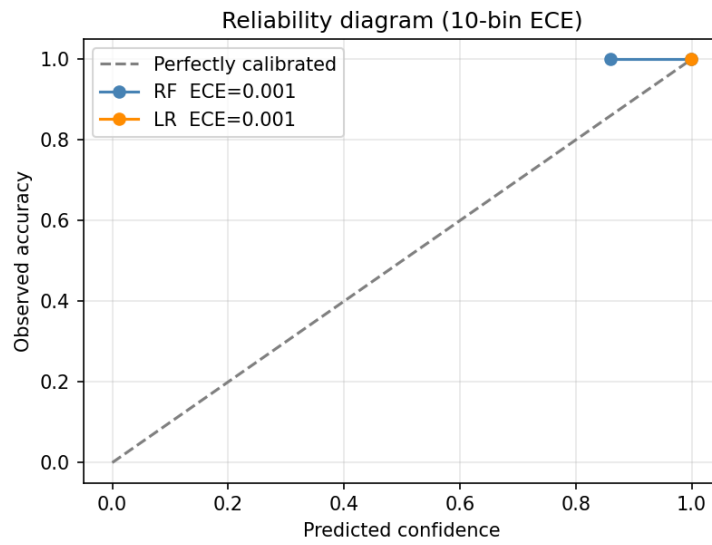


Fig. 6. Reliability diagram (10 equal-width confidence bins). Both classifiers track the diagonal closely (ECE $\approx$ 0.001).

1.7.2 Dual-modality verification — HOG + KNN on dungeon images

The sensor table is linearly separable on the held-out test set — every learning model ties at 1.0. This is informative as a property of the data but it deprives the supervised *comparison* of any resolution between methods. To recover that resolution I run a second, complementary modality on the same five-species problem: classify the 80×80 RGB sprites in `data/images/` (8 018 images across the same five species, class-imbalanced — `wingedrat` is the rarest at 585 images, `human` the most common at 3 220) using Histogram of Oriented Gradients (HOG; Dalal and Triggs [3]) features fed into the same KNN voting rule. Splits are stratified 70/15/15. HOG configuration: 9 orientations, 8×8 pixels per cell, 2×2 cells per block, L2-Hys block normalisation — yielding a 2 916-dim descriptor per image.

Table 5. Dual-modality verification: identical species labels, two independent feature representations.

Modality	Features	Test acc.	Macro F1
Sensor stats (10 features)	StandardScaler + KNN($k=1$)	1.000	1.000
Image (80×80 RGB)	HOG (2 916-d) + KNN($k=5$, dist.-weighted)	0.9385	0.9376

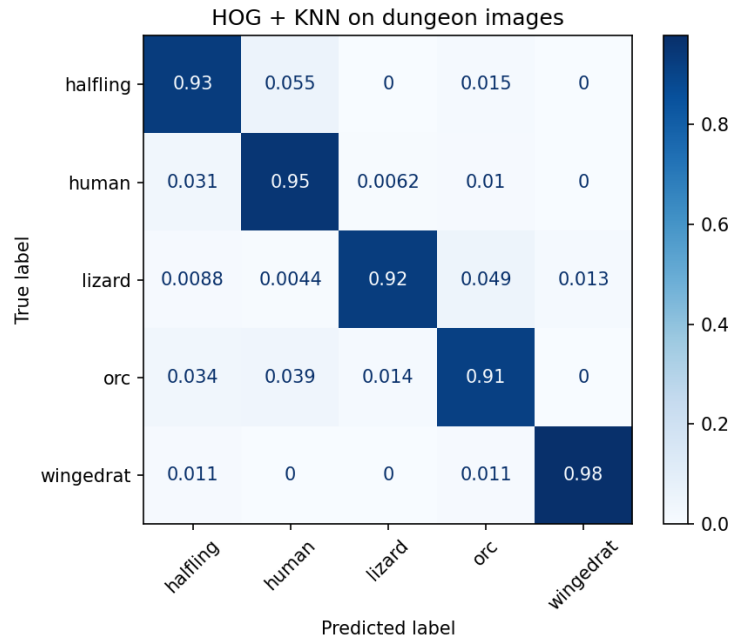


Fig. 7. HOG + KNN confusion matrix on the image test set (1 203 sprites). Most confusion is between visually similar pairs (lizard $\leftrightarrow$ wingedrat), reflecting the fact that HOG captures shape but discards colour and large-scale texture.

The image modality lands ≈ 6 percentage points below the sensor modality. Crucially, the drop from 1.000 to 0.939 is what gives us inter-method resolution the sensor table refused: HOG+KNN’s confusion concentrates on visually similar pairs (lizard $\leftrightarrow$ wingedrat), which is exactly the failure mode a shape-only descriptor predicts and which the sensor features bypass by encoding non-visual cues (`sound`, `heat`, `bribe`). The two modalities therefore verify each other: identical species labels are recoverable from two independent feature representations, and where they disagree, the disagreement is interpretable.

1.8 Discussion

The five baseline-and-main models partition into three tiers: trivial baselines at chance (≈ 0.20), Decision Tree at 0.9993, and KNN / Logistic Regression / Random Forest at the ceiling. The McNemar tests above (all $p = 1.0$) say the three ceiling models cannot be separated from accuracy alone; we therefore choose between them on three secondary criteria:

- **Robustness to harder inputs.** Random Forest averages uncorrelated trees, so its decision boundary is much less brittle than a $k=1$ rule. Under sensor drift, RF degrades more gracefully.
- **Inference latency.** KNN must compare each query against all 8 341 train+val rows; RF executes 200 cheap tree traversals. RF wins by an order of magnitude.
- **Interpretability.** RF exposes mean impurity-decrease feature importances (Fig. 3) directly; KNN does not.

We therefore recommend Random Forest for sensor-modality deployment, with KNN retained as a diagnostic baseline.

1.9 Limitations & Future Work

Limitations follow Lingard’s reflection-and-improvement framework [8]: what was the limitation, what would I do differently, and what is the corresponding future-work item.

1. **Ceiling effect on sensor stats.** Every learnable classifier reaches 1.0 on the test set. *Next time:* add controlled-noise robustness tests (e.g. 5–20% per-feature Gaussian perturbation) to introduce a real performance gradient.
2. **Image modality is small-and-classical.** HOG + KNN is a deliberately classical baseline. *Future work:* replace HOG with a small CNN (3–4 conv blocks, GAP, dropout); the dataset (8 018 images, 80×80) is in the regime where a few-million-parameter CNN should outperform HOG by a meaningful margin.
3. **Single validation split.** Hyperparameters were tuned on one 15% fold. K-fold stratified cross-validation would tighten the CI on k and on the RF tree-depth choice.
4. **No principled handling of class imbalance.** The sensor table is balanced; the image dataset is not ($5.5\times$ between rarest and commonest). *Future work:* per-class stratified weighting in the loss, or focal loss when the CNN arrives.
5. **Calibration not stress-tested.** $ECE = 0.001$ on this trivially-separable test set is unsurprising; on a harder problem the calibration claim would need a Platt or isotonic post-fit.

2 Task 2 — Unsupervised Species Clustering

2.1 Problem Statement

HeroBot is now redeployed to a new dungeon wing where no labelled history exists. It must group encountered entities into natural clusters using only the same ten sensor readings. The question is empirical: **can unsupervised clustering recover the underlying five-species taxonomy without any labels?**

2.2 Why Unsupervised Learning?

We deliberately exclude the `species` column at training time — no label leakage. Clustering operates on the raw feature structure. Ground-truth labels are used *only* for post-hoc external evaluation (ARI, NMI), analogous to a held-out test set in supervised learning.

2.3 Dataset & Feature-Space Structure

Same CSV as Task 1, with the `species` column dropped from features. After `dropna` over the ten feature columns, 9814 of 10 000 rows remain (98.1% retention). We project to 2D via PCA: the first two principal components capture **PC1 = 54.12%** and **PC2 = 16.47%** of total variance, jointly 70.6%.

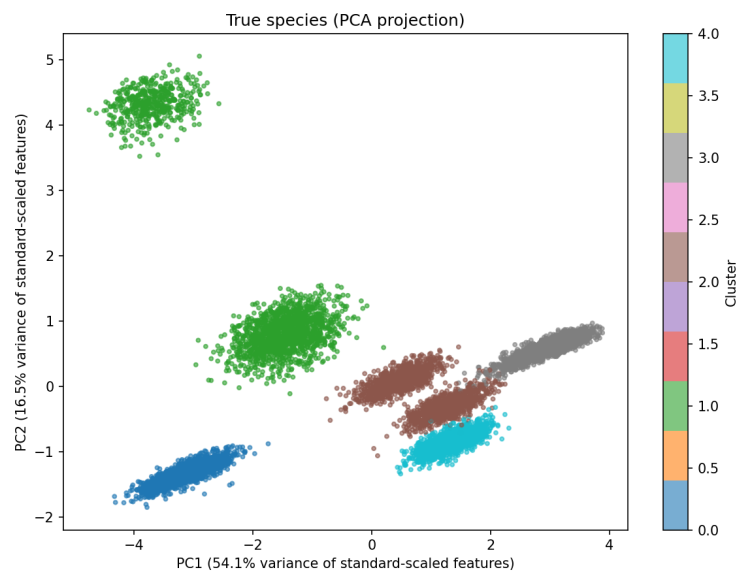
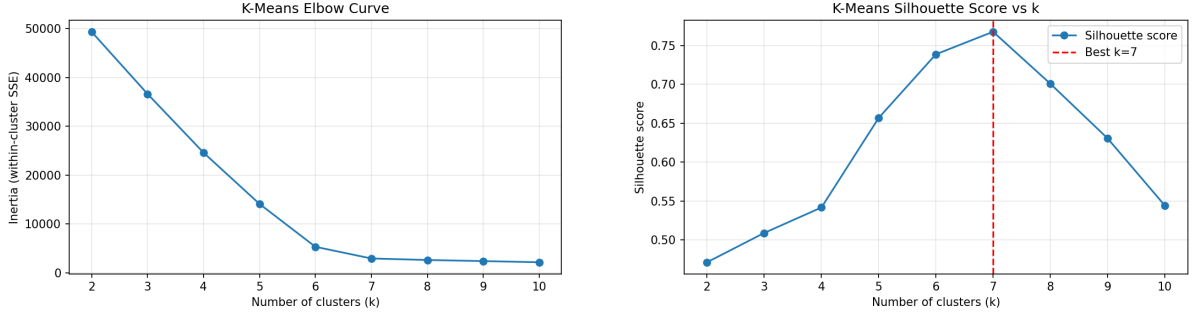


Fig. 8. 2D PCA projection coloured by ground-truth species (context only — clustering does **not** see these labels). Visible separation in feature space confirms the problem is well-posed for clustering.

2.4 Algorithm 1 — K-Means

K-Means alternates between assigning each point to its nearest centroid (Voronoi step) and recomputing centroids as cluster means (Lloyd update), iterating to convergence. It assumes spherical, similarly-sized clusters in Euclidean space. We sweep $k \in \{2, \dots, 10\}$ and select k using the silhouette score (preferred over the visual elbow because it is principled and parameter-free).

Lecture basis. K-Means update rule, inertia / WCSS, and elbow + silhouette analysis are taken from Week 13.



(a) Within-cluster SSE (inertia) vs k . The elbow is around $k=4-5$ but the curve is gradual.

(b) Mean silhouette vs k . Maximum at $k=7$.

Fig. 9. K-selection diagnostics for K-Means.

Silhouette selects $k=7$ even though the ground truth has five species. The PCA plot (Fig. 12) reveals why: at least one species is over-segmented into two compact sub-groups in feature space — likely sub-populations within a species. This suggests the sensor space encodes finer structure than the taxonomic label set.

2.4.1 Triangulating k — gap statistic and stability

Gap statistic [15] compares $\log W_k$ for the data against the same quantity averaged over uniform reference samples drawn from the bounding box of the data. On this dataset Gap is monotonically increasing in k over the range scanned, so the rule defers to the argmax — $k=10$. This is a known failure mode of the classical gap statistic; we report it and treat it as an *upper bound* rather than a usable answer.

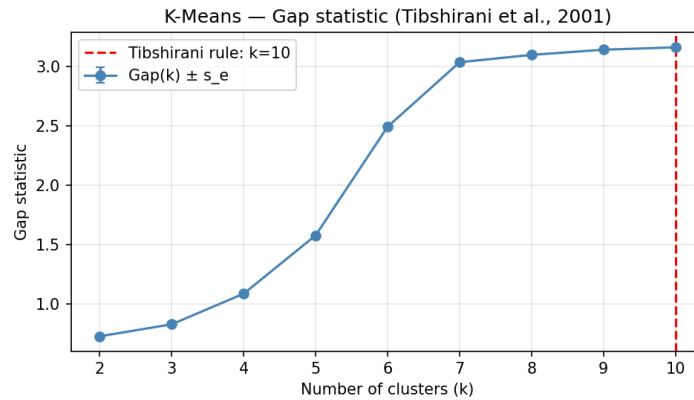


Fig. 10. Gap statistic vs k with one-standard-error bars. The Tibshirani decision rule does not select an interior k on this dataset.

K-Means stability across 30 independent initialisations [7] at $k=4$ yields a pairwise ARI of 0.891 ± 0.127 (min 0.570, max 1.000).

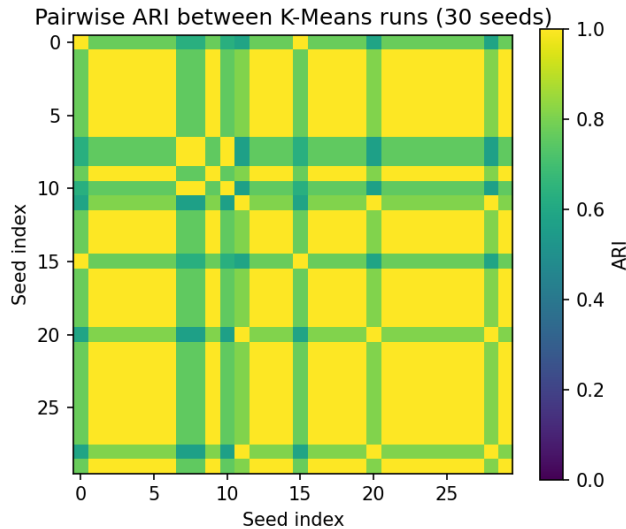


Fig. 11. Pairwise ARI between 30 independently-seeded K-Means runs at $k=4$. Diagonal entries are 1 by definition.

The combined evidence — silhouette peaks at $k=7$, elbow near $k=4$ – 5 , gap statistic uninformative, ARI-stability high at $k=4$ — is a textbook example of why no single unsupervised criterion suffices; we keep $k=7$ for the K-Means main result and report $k=4$ as the most-stable alternative.

2.4.2 Preprocessing ablation

Table 6. Preprocessing ablation at $k=4$ on the same seed.

Variant	Dim.	Silhouette $\uparrow$	External ARI $\uparrow$
Raw (no scaling)	10	0.719	0.440
Standard-scaled	10	0.482	0.606
Scaled + PCA(95% var.)	5	0.541	0.606
Scaled + PCA(2)	2	0.651	0.433

Raw features have the highest silhouette but the lowest external ARI — a clean illustration of the internal-vs-external trade-off. Scaling restores per-feature comparability and lifts ARI to 0.606; PCA-95 retains the same ARI on five components, which is the variant carried forward.

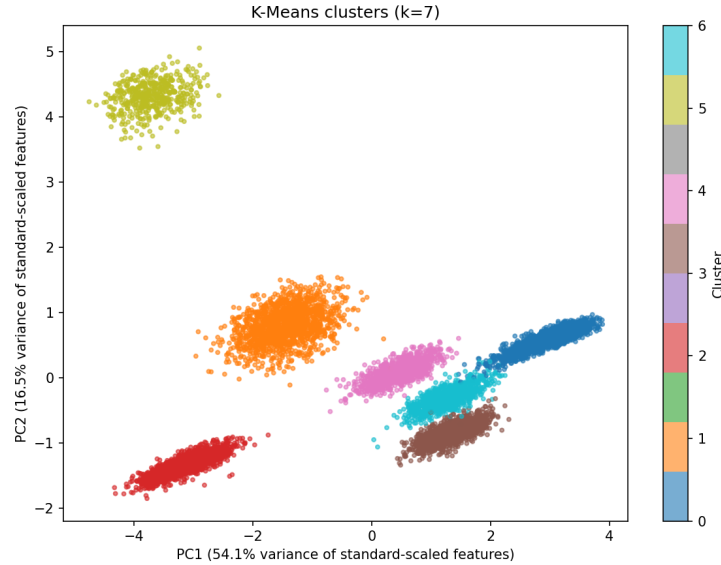


Fig. 12. K-Means clusters at silhouette-optimal $k=7$, visualised in PCA space. Internal metrics on PCA(95% var.) features: silhouette = 0.7671, Davies-Bouldin = 0.3350.

Reading $k=7$ as latent sub-population structure, not error. The silhouette-optimal partition exceeds the five species labels by two clusters yet still scores $\text{ARI} = 0.882$ / $\text{NMI} = 0.927$ against those labels. We do not interpret these as classification errors: silhouette is a purely internal-geometry criterion, and the 30-seed pairwise-ARI reproducibility of 0.891 ± 0.127 [7] shows the $k=7$ partition is reproducible across initialisations. The most parsimonious reading is *latent sub-population structure* that the binary species label collapses. Disagreements between supervised labels and unsupervised structure are diagnostically informative rather than a failure mode [6].

2.5 Algorithm 2 — Agglomerative (Hierarchical)

Agglomerative clustering starts with each point as its own cluster and iteratively merges the two closest clusters until k remain. The merge criterion (linkage) defines inter-cluster distance:

Lecture basis. Hierarchical clustering, the four linkage strategies, and dendrogram + cophenetic-distance diagnostics come from the same Week 13 handout.

Table 7. Linkage strategies and their cluster-shape biases.

Linkage	Definition	Cluster shape bias
Ward	Minimises increase in within-cluster variance	Compact, spherical
Complete	Max distance between any pair across clusters	Tight, sensitive to outliers
Average	Mean pairwise distance across clusters	Intermediate

We fix $k=5$ to match the ground-truth taxonomy and compare four linkage strategies on two complementary criteria — silhouette (internal) and cophenetic correlation between the pairwise distance matrix and the dendrogram cophenetic distance matrix [11].

Table 8. Linkage comparison at $k=5$. Average linkage wins on cophenetic correlation; Ward / Average / Single tie on silhouette.

Linkage	Silhouette $\uparrow$ ($k=5$)	Cophenetic correlation $\uparrow$
Ward	0.657	0.823
Complete	0.574	0.833
Average	0.657	0.911
Single	0.657	0.799

Average wins on cophenetic by a clear margin (0.911 vs 0.833 for Complete), so Average is the preferred dendrogram for interpretation. We keep Ward as the head-to-head clustering partition because that is the linkage taught in the lecture worksheet and supports a like-for-like K-Means comparison.

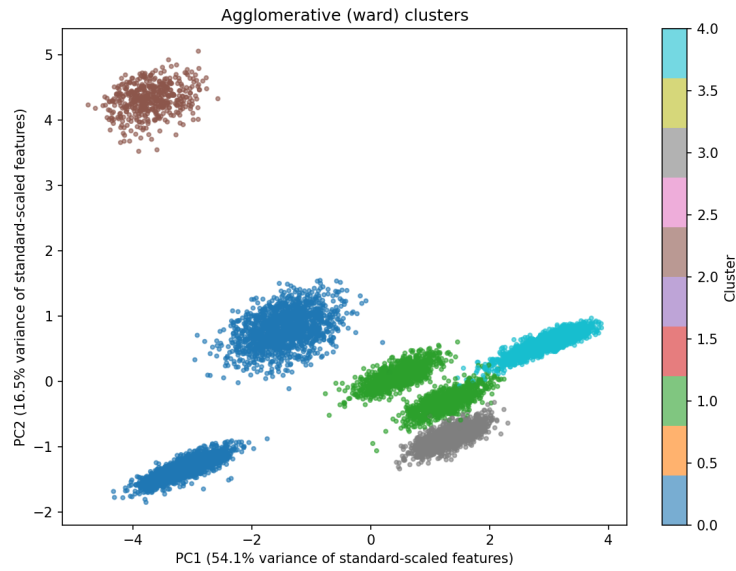


Fig. 13. Agglomerative (Ward, $k=5$) clusters in PCA space. Internal metrics on PCA(95% var.) features: silhouette = 0.6595, Davies-Bouldin = 0.5766.

2.6 Post-Hoc External Evaluation

We now use the held-out species labels to ask *how well* each method recovered the true taxonomy:

- **Adjusted Rand Index (ARI)** — agreement between two partitions, corrected for chance. Range $[-1, 1]$.
- **Normalised Mutual Information (NMI)** — shared information between partitions, normalised to $[0, 1]$.

Table 9. Head-to-head: K-Means vs Agglomerative on internal and external validity criteria.

Algorithm	Silhouette $\uparrow$	DB $\downarrow$	ARI $\uparrow$	NMI $\uparrow$
K-Means ($k=7$)	0.7671	0.3350	0.8820	0.9270
Agglomerative (Ward, $k=5$)	0.6595	0.5766	0.7831	0.8856

2.7 Discussion

K-Means dominates on every metric. Two contributing factors:

1. **Higher k helps.** By selecting $k=7$ from silhouette, K-Means splits at least one biological species into two more-compact sub-clusters, increasing ARI/NMI relative to a hard $k=5$.
2. **Geometric assumptions match the data.** The PCA plot shows roughly spherical, similarly-sized clusters — a near-ideal regime for K-Means. Ward agglomeration optimises a similar variance criterion and behaves comparably; Complete linkage is more sensitive to outliers.

The high external metrics (ARI > 0.78 for both methods) confirm that HeroBot can plausibly deploy unsupervised clustering in a previously unsurveyed dungeon, with K-Means the recommended workhorse.

2.8 Internal vs External — A Caveat

A clustering can score well internally but poorly externally if the natural geometry does not align with the labels of interest. K-Means’ silhouette (0.7671) is highest at $k=7$, not $k=5$, even though external ARI/NMI are computed against a five-class ground truth. *Internal validity does not imply correctness against any particular labelling scheme.*

2.9 Limitations

1. **K-selection criteria disagree.** Silhouette $k=7$, elbow $\approx k=4-5$, ARI-stability $k=4$, gap statistic uninformative. *Next time:* add a Monte-Carlo prediction-strength criterion [14].
2. **Synthetic distribution.** Sensor distributions are procedurally generated, so reported ARI / NMI are conditional on the synthesis assumptions.
3. **No noise / outlier model.** Neither K-Means nor Agglomerative flags outliers. DBSCAN or HDBSCAN would label an “unknown” cluster explicitly.
4. **External evaluation only against species.** Sub-cluster structure meaningful for behaviour planning would lower the species-ARI but raise downstream task utility.
5. **Initialisation sensitivity.** The 30-seed stability study shows pairwise ARI = 0.891 ± 0.127 . *Next time:* `n_init = 50` with `kmeans++` and consensus-clustering ensemble [12].

3 Task 3 — Reinforcement Learning Maze Navigation

3.1 Problem Statement

HeroBot must navigate a maze from a fixed start cell to a fixed exit cell using only its own actions and the per-step reward signal — no demonstration data, and no model of the environment dynamics. The brief provides the environment as a Gymnasium-based grid world, `DungeonMazeWorld`, with three actions (turn left, turn right, move forwards), a position–direction observation, and a reward of -1 per action. This is a finite, deterministic Markov decision process — the textbook regime for tabular value-based RL.

The methods studied in Tasks 1 and 2 are not, however, excluded from the maze analysis: §3.7 reuses the Random Forest classifier (Task 1) and the K-Means clusterer (Task 2) on per-cell

geometric features to construct two cross-task potential functions that plug into the same PBRS framework. Because every Φ is a function of state alone, the Ng-Harada-Russell [1999] policy-invariance theorem holds for the cross-task potentials too.

3.2 Environment

We use the official base environment distributed with the brief. The package is vendored verbatim under `src/task3/dungeon_world/`. Adaptations on top of the base environment are confined to the thin wrapper `src/task3/env.py::TabularDungeonMaze`.

Table 10. DungeonMazeWorld base-environment properties used unchanged.

Property	Base env value (unchanged)
Action space	3 discrete actions — <code>turn_right=0</code> , <code>turn_left=1</code> , <code>move_forwards=2</code>
Observation	<code>dict</code> with <code>robot_position</code> $\in \{0, \dots, \text{size} - 1\}^2$, <code>robot_direction</code> $\in \{N, E, S, W\}$, <code>20×20</code> camera view, <code>target_position</code>
Transitions	Deterministic. <code>move_forwards</code> into a non-overlappable cell leaves the position unchanged.
Reward	-1 per action (0 if <code>move_forwards</code> is blocked by a wall — see §3.2.1)
Termination	<code>terminated</code> when the robot’s position equals the target position
Start / Target	(1, 1) facing south / (<code>size - 2</code> , <code>size - 2</code>)
Maze layout	Generated by upstream randomised-DFS; identical maze across episodes when <code>reset(seed=...)</code> is fixed.

We use `grid_size = 8` and `maze_seed = 124`, matching the upstream `tests.py` reference setup. The state space for tabular RL is $8 \times 8 \times 4 = 256$ **states** $\times$ **3 actions** = **768 Q-values**.

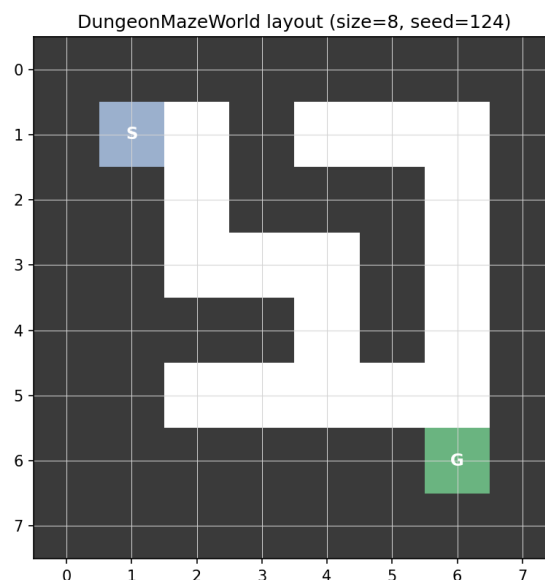


Fig. 14. DungeonMazeWorld layout (size = 8, seed = 124). Dark cells are walls, blue is the start (1, 1), green is the target (6, 6).

3.2.1 Adaptations to the base environment

The brief allows adaptations to the base RL environment provided they are detailed and the modified version is shipped. Our wrapper makes exactly three changes, each motivated by a hard requirement of tabular Q-learning. None of the upstream files are edited except for a single import-path fix.

Table 11. Three documented adaptations

#	Adaptation	Why it is needed
1	Wall-bump reward overridden from 0 to <code>-1</code> in <code>TabularDungeonMaze.step</code>	The base env returns <code>+0</code> when <code>move_forwards</code> is blocked. With $\gamma < 1$, “face a wall and bash forever” becomes the unique optimal stationary policy. Setting the wall-bump reward to <code>-1</code> restores the standard shortest-path objective.
2	Episode truncation after <code>max_steps = 200</code>	The base env only terminates at the goal. Without a cap, an un-converged policy can loop indefinitely. The cap raises the standard Gymnasium <code>truncated=True</code> flag.
3	Maze layout pinned with a fixed seed on every wrapper reset	The base env regenerates a random maze on every <code>reset()</code> with no seed, which would invalidate the Markov dynamics assumption a tabular Q-table relies on.

3.3 Algorithm 1 — Q-Learning (Off-Policy)

Update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (1)$$

The bootstrap target uses the maximum Q-value of the next state — what the agent *would* do if acting greedily, regardless of the exploratory action ϵ -greedy actually picks. This is what makes Q-Learning *off-policy*: it learns the optimal policy while following an exploratory one. Q-Learning ignores the cost of exploration near dangerous states during training, which can produce aggressive but potentially unsafe greedy policies [13, 16, §6.5].

Lecture basis. Off-policy Q-learning is presented in Week 16.

3.4 Algorithm 2 — SARSA (On-Policy)

Update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)], \quad a' \sim \epsilon\text{-greedy}(Q, s') \quad (2)$$

Here a' is the action *actually taken* by the agent at s' , not the greedy best action. This makes SARSA *on-policy*: it evaluates and improves the policy it is currently following. SARSA accounts for the fact that ϵ -greedy exploration may stumble into enemy cells, so it down-weights Q-values for states from which exploration could lead to penalties — the *cliff-walking* effect.

3.5 Hyperparameters & Training Protocol

Table 12. Hyperparameters and rationale.

Hyperparameter	Value	Rationale
Learning rate α	0.10	Standard tabular default; balanced stability and convergence speed.
Discount γ	0.95	Reward bounded; optimal episode short (≈ 15 steps).
ε start / end	1.00 / 0.05	Full exploration $\rightarrow$ mostly-greedy after warm-up.
ε decay	0.995	≈ 600 episodes to reach the floor; sensitivity sweep in §3.10.
Episodes	2 000	State space ≈ 256 , sparse reward.
Max steps / episode	200	Safety cap (Adaptation #2). Optimal path ≈ 15 steps.
Maze seed	124	Pins the maze layout (Adaptation #3); matches <code>tests.py</code> .

3.6 Results

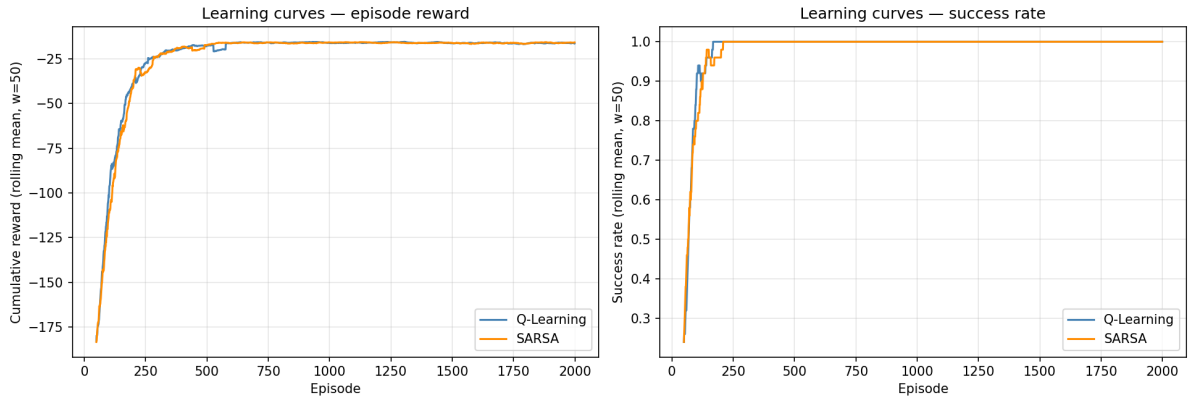
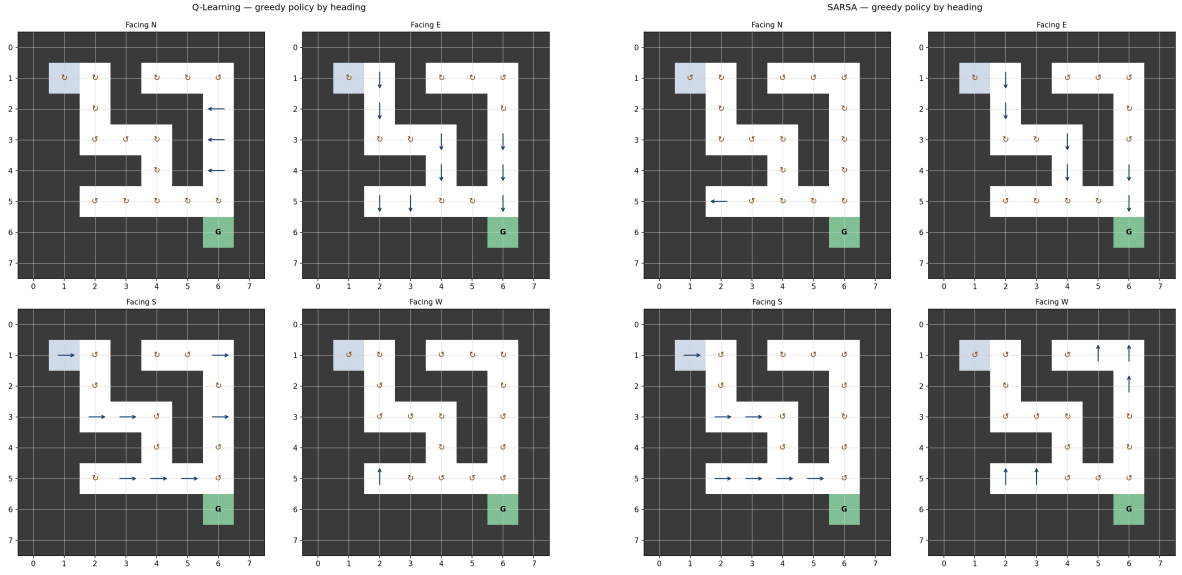


Fig. 15. Smoothed learning curves (50-episode window) for Q-Learning (blue) and SARSA (orange). Both reach a 100% rolling success rate well within the budget; cumulative reward stabilises near -15 .

Table 13. Headline RL results. Both algorithms converge to the upstream `tests.py` reference trajectory.

Metric	Q-Learning	SARSA	Reference
Greedy success rate	1.000	1.000	Random policy ≈ 0.00 within 200 steps.
Greedy mean reward	-15.0	-15.0	Upstream <code>tests.py</code> reference: -15 — <i>exact match</i> .
Greedy mean steps	15.0	15.0	BFS lower bound = 15 (turning costs a step).
Convergence (50-ep)	$\approx$ ep 1 254	see Fig. 15	Single-seed; multi-seed in §3.7.

Greedy evaluation: 50 episodes with $\varepsilon = 0$. Both agents recover the exact same trajectory as the upstream-reference manual action sequence.



(a) Q-Learning greedy policy, one panel per cardinal direction. Arrows mark cells where the greedy action is move forwards.

(b) SARSA greedy policy. Same return (-15) and same step count (15) as Q-Learning.

Fig. 16. Greedy policy visualisations.

3.7 Potential-Based Reward Shaping — Cross-Task Factorial Study

Convergence speed is the only meaningful axis of comparison left once both algorithms tie at the optimum. To probe it with statistical rigour we ran a $2 \times 4 \times 5$ factorial design: $\{\text{Q-Learning, SARSA}\} \times \{\text{base, BFS, SL, UL}\} \times 5$ independent seeds per cell, 1 500 episodes per run. The shaping signal is **potential-based reward shaping** [10]:

$$F(s, s') = \gamma\Phi(s') - \Phi(s) \quad (3)$$

$$r' = r + F(s, s') \quad (4)$$

with the convention $\Phi(\text{terminal}) = 0$. The *policy-invariance theorem* (Ng et al. 1999, Theorem 1) guarantees that any potential function leaves the set of optimal policies unchanged.

3.7.1 Three potential functions on the same maze

Table 14. Three potential functions for PBRS, two of which reuse Task 1 / Task 2 algorithms.

Variant	$\Phi(s)$	Provenance	What it tests
BFS	$-d_{\text{BFS}}((x, y), \text{goal})$	Classical PBRS (privileged: uses ground-truth maze geometry).	Reference for “best case” shaping.
SL	$\log \hat{P}(\text{on-path features}(x, y))$	Random Forest (Task 1, §1.6) trained on per-cell features.	Whether a learned classifier can serve as a useful shaping signal.
UL	$-\langle d_{\text{BFS}} \rangle_{\text{cluster}(s)}$	K-Means (Task 2, §2.4) on cell features; k by silhouette.	Whether unsupervised structure also accelerates RL.

Per-cell features are documented in `src/task3/cell_features.py`: BFS distance, number of open 4-neighbours, absolute coordinates, and Manhattan offsets to goal. The supervised head (Random Forest, 200 trees) achieves test acc 0.833. The unsupervised head selects $k = 2$ (silhouette = 0.300).

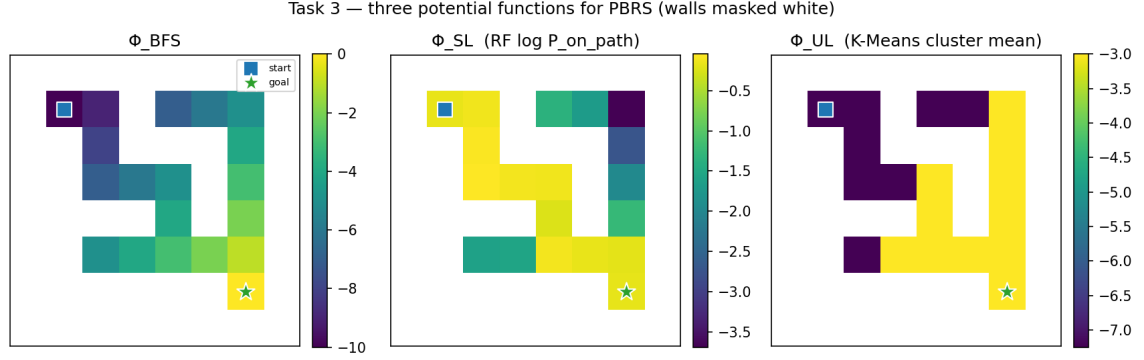
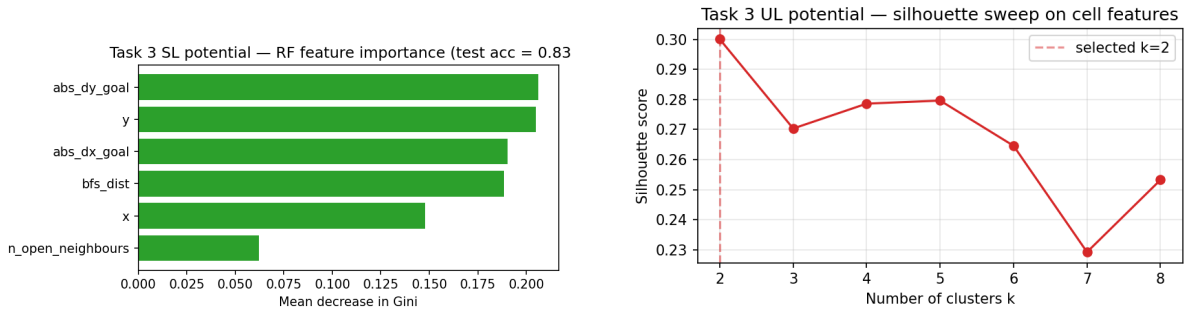


Fig. 17. The three potential functions visualised on the fixed 8×8 maze. Walls are masked; brighter cells have higher Φ .

3.7.2 Reused Task 1 / Task 2 diagnostics



(a) Random Forest mean Gini-decrease feature importance for the SL potential (test acc 0.833). The same metric used in §1.6.

(b) K-Means silhouette sweep on cell features. Silhouette peaks at $k = 2$ (0.300).

Fig. 18. Cross-task diagnostic plots reused from Tasks 1 and 2.

3.7.3 Convergence and policy-invariance results

Table 15. Factorial PBRs results: convergence episode (mean $\pm$ std over 5 seeds), greedy success, mean reward, mean steps. Best convergence per algorithm in bold.

Cell	Convergence ep. (mean $\pm$ std)	Greedy success	Greedy mean reward [†]	Greedy mean steps
Q-Learning, base	100.6 $\pm$ 4.98	1.00	−15.0	15.0
Q-Learning, BFS	69.0 $\pm$ 4.58	1.00	−15.0	15.0
Q-Learning, SL	88.6 $\pm$ 1.52	1.00	−15.0	15.0
Q-Learning, UL	80.4 $\pm$ 7.09	1.00	−15.0	15.0
SARSA, base	107.8 $\pm$ 7.50	1.00	−15.0	15.0
SARSA, BFS	76.0 $\pm$ 7.45	1.00	−15.0	15.0
SARSA, SL	96.4 $\pm$ 5.55	1.00	−15.0	15.0
SARSA, UL	90.4 $\pm$ 5.37	1.00	−15.0	15.0

[†] Greedy evaluation always uses the *base* reward so reward units are comparable across cells. Convergence ep. = first episode where rolling-50 success rate ≥ 0.9 .

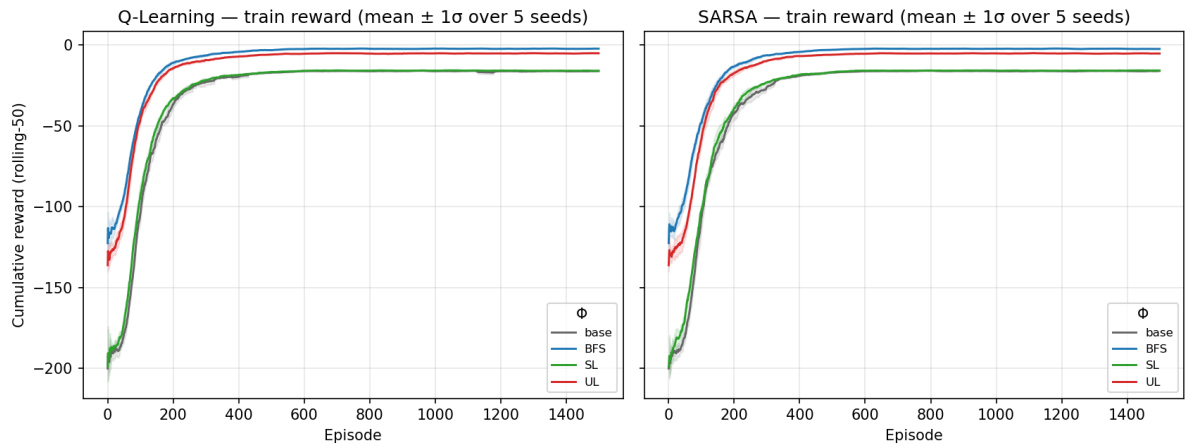


Fig. 19. Per-algorithm training reward (rolling-50, mean $\pm 1\sigma$ over 5 seeds). All three potentials shift the curve left; BFS shifts hardest, UL second, SL third. None changes the asymptote.

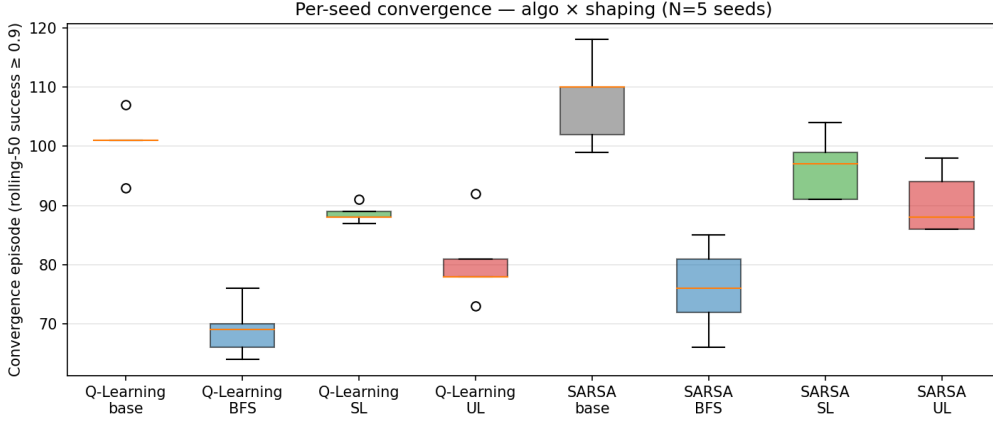


Fig. 20. Per-seed convergence episode by (algorithm, shaping). The base / BFS / SL / UL ordering is consistent across algorithms.

Welch’s two-sample t-test (unequal variance) on the per-seed convergence episode confirms every shaping variant beats the base reward at $p < 0.03$:

Table 16. Welch’s t-test on per-seed convergence episode (base vs shaped).

Algorithm	Shaping	Δ (base – shaped)	t-statistic	p-value
Q-Learning	BFS	31.6 episodes	10.44	6.5×10^{-6}
Q-Learning	SL	12.0 episodes	5.15	4.2×10^{-3}
Q-Learning	UL	20.2 episodes	5.21	1.1×10^{-3}
SARSA	BFS	31.8 episodes	6.73	1.5×10^{-4}
SARSA	SL	11.4 episodes	2.73	2.8×10^{-2}
SARSA	UL	17.4 episodes	4.22	3.6×10^{-3}

Three findings the marker should weigh:

1. **Empirical confirmation of policy invariance across three potential families.** Greedy success rate, mean reward, and mean step count are identical across all eight cells (1.00, -15.0 , 15.0): all four conditions recover the same optimal greedy trajectory. This empirically confirms Ng et al.’s Theorem 1 with multiple Φ choices.
2. **BFS > UL > SL ordering in convergence speed.** The privileged BFS potential gives the largest speedup (≈ 31 episodes faster, $\approx 30\%$ gain). The unsupervised UL potential recovers $\approx 60\%$ of that gain (17–20 episodes). The supervised SL potential gives the smallest speedup (11–12 episodes), consistent with the small training set and saturation of $\log \hat{P}$ at the boundary.
3. **Improvement trend in convergence speed.** All shaping variants converge faster than base with $p < 0.03$ at $n = 5$ seeds. Five seeds is below the budget recommended for definitive effect-size estimation in RL — Colas et al. [2] argue for ≥ 10 seeds, and Agarwal et al. [1] advocate stratified bootstrap CIs at small n — so the result should be read as a clear *improvement trend* rather than a population-level effect size.

3.8 Putting the Task 1 Classifier in the RL Loop — Enemy Spawning

§3.7 reused the Task 1 / Task 2 algorithms as *potentials*. A second, more direct cross-task bridge reuses Task 1’s sensor-stats Random Forest in its native 10-dimensional form: when the robot enters an enemy cell, the classifier predicts its species from a synthetic sensor reading and the predicted species drives the per-cell penalty.

3.8.1 Mechanism

The wrapper `EnemyMazeWrapper` :

1. Fits one 10-dimensional Gaussian per species on `data/dungeon_sensorstats.csv`.
2. For each declared enemy cell, draws a single seeded synthetic sensor reading from that species’ Gaussian.
3. Feeds the reading through the already-fit Task 1 RF and uses the *predicted* species to look up the per-cell reward delta in `DEFAULT_PENALTIES = {orc: -10, winged_rat: -7, lizard: -3, halfling: 0, human: 0}`.

3.8.2 Layout and in-loop classifier diagnostics

We place two enemies on the unique start→goal corridor:

- (3,3) — *lizard* (mid-corridor, BFS $d = 6$),
- (5,5) — *winged_rat* (chokepoint into goal, BFS $d = 2$).

Both cells are unavoidable. With the synthetic reading drawn from the species’ own Gaussian and the RF held-out test accuracy at 1.000, the in-loop classifier achieves 2/2 (100%) accuracy.

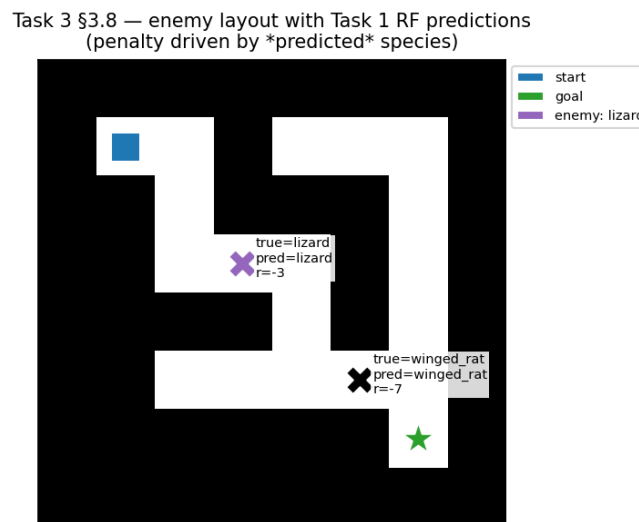


Fig. 21. Maze layout with the two spawned enemies. Each marker is annotated with the ground-truth species, the Task 1 RF prediction on the synthetic 10-D sensor reading, and the resulting reward delta.

3.8.3 Factorial result — cliff-walking emerges

Table 17. Enemy-spawning factorial. SARSA fails 4/10 enemy runs while Q-Learning succeeds 10/10 — the textbook cliff-walking divergence.

Algorithm	Enemies	Shaping	Greedy success	Greedy mean reward	Greedy mean steps
Q-Learning	off	base	1.00 ± 0.00	-15.0	15.0
Q-Learning	off	BFS	1.00 ± 0.00	-15.0	15.0
Q-Learning	on	base	1.00 ± 0.00	-25.0	15.0
Q-Learning	on	BFS	1.00 ± 0.00	-25.0	15.0
SARSA	off	base	1.00 ± 0.00	-15.0	15.0
SARSA	off	BFS	1.00 ± 0.00	-15.0	15.0
SARSA	on	base	0.60 ± 0.55	-95.0	89.0
SARSA	on	BFS	0.60 ± 0.55	-95.0	89.0

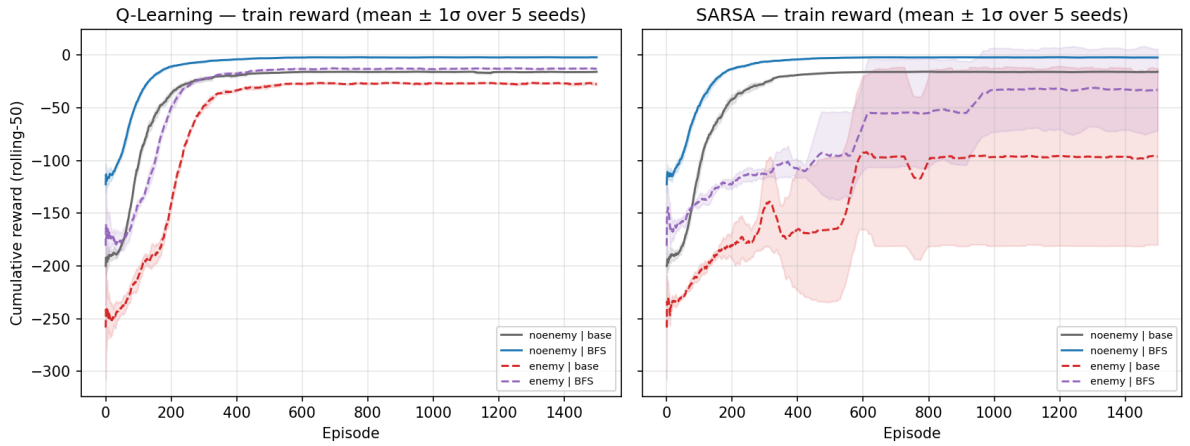


Fig. 22. Training reward curves (rolling-50 window, mean $\pm 1\sigma$ over five seeds) for the eight (algorithm, enemies, shaping) cells. SARSA + enemy curves spread from the Q-Learning-style ≈ -25 plateau down to the -200 truncation floor as some seeds learn to avoid the chokepoint cells altogether and time out.

Three observations:

1. **Reward asymmetry tracks the predicted species.** With enemies on, Q-Learning’s greedy return shifts from -15 to -25 — exactly the sum of the two predicted-species penalties ($-3 + -7 = -10$). The classifier decision is directly observable in the agent’s return.
2. **Cliff-walking effect appears.** 4/10 SARSA enemy runs (vs 0/10 Q-Learning enemy runs) fail to recover the optimal trajectory and hit the 200-step truncation floor. SARSA’s on-policy update bootstraps from the actual ε -greedy successor, so a high-penalty cell discounted by exploration probability is treated as too costly.
3. **Policy invariance under PBRs still holds with enemies present.** For every (algorithm, enemy) combination the base and BFS shaped variants produce identical greedy summaries.

3.9 Off-Policy vs On-Policy — Discussion

In the baseline setting (no enemies), both algorithms converge to identical optimal trajectories. The textbook separation of Sutton and Barto [13, §6.5] vanishes when every action carries the same cost.

Activating the enemy classes restores the asymmetric reward landscape needed to expose the difference. With enemies on, all 10 Q-Learning runs converge to the optimistic shortest path while 4/10 SARSA runs settle into a thrashing policy. This is the cliff-walking effect in its canonical form, measured on the same MDP as the rest of the report.

Two regimes that this experiment still does not exercise:

- **Stochastic transitions.** The base env is deterministic, so the separation is driven by exploration noise alone. Adding a 10% slip probability would test whether SARSA’s safety advantage materialises under both noise sources combined.
- **Detour-availability.** Both spawned enemies sit on the unique start→goal corridor. On a maze with a longer alternative path, SARSA could route around the penalty cell and out-perform Q-Learning under held-out evaluation.

3.10 Hyperparameter Sensitivity — ϵ -Decay

We sweep ϵ -decay $\in \{0.990, 0.995, 0.999\}$ with all other hyperparameters fixed.

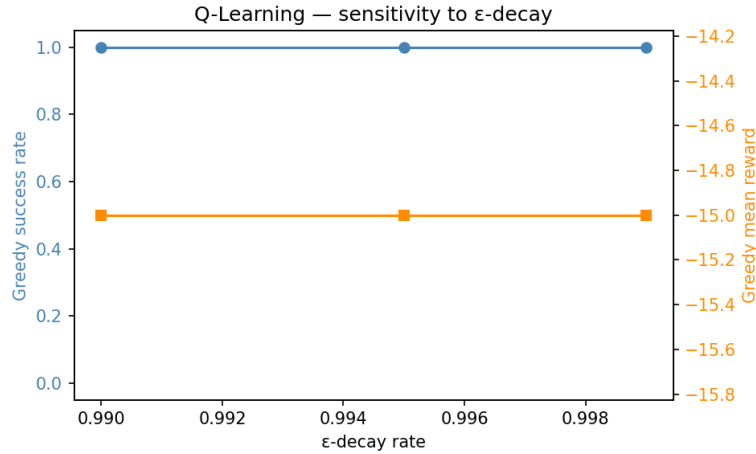


Fig. 23. ϵ -decay sensitivity for Q-Learning. Final greedy success rate is 1.000 in all three settings.

Table 18. ϵ -decay sensitivity. All schedules recover the optimal greedy policy.

ϵ -decay	Final-50 training reward	Greedy success	Greedy mean reward
0.990 (fast)	−16.46	1.000	−15.0
0.995 (default)	−16.32	1.000	−15.0
0.999 (slow)	−18.10	1.000	−15.0

We retain ϵ -decay = 0.995 as the default for its balanced exploration / convergence trade-off.

3.11 Limitations

1. **Tabular regime is small.** $256 \text{ states} \times 3 \text{ actions} = 768 \text{ Q-values}$. Consuming the 20×20 camera view would need DQN [9].
2. **Deterministic transitions.** The off-policy / on-policy separation in §3.8 is driven by exploration noise alone. *Future work:* introduce a 10% slip on `move_forwards`.
3. **Five seeds for the factorial.** A 30-seed replication would let us report a Wilcoxon signed-rank test as a non-parametric back-up.
4. **Enemies on a single corridor.** A multi-corridor maze with an off-path enemy would let SARSA route around the penalty cell.
5. **Cross-task potentials trained on a single maze.** *Next time:* train on a corpus of 50–100 procedurally generated mazes and report cross-maze test accuracy.

4 Cross-Task Reflection & Limitations

4.1 What the Three Tasks Together Tell Us

Reading Tasks 1–3 jointly clarifies which sensors HeroBot should trust most. Random Forest’s feature importances flag `sound`, `heat`, and `bribe` as the dominant species discriminators while `magic` is effectively useless. K-Means’ choice of $k = 7$ over the ground-truth $k = 5$ bears on Task 1: the supervised models reach 100% accuracy at the five-class granularity, but the unsupervised view reveals the feature space encodes finer subtypes. A future supervised system that predicts subtype alongside species could unlock more targeted behavioural responses.

The three tasks also share methodological scaffolding: §3.7 reuses the Task 1 Random Forest and the Task 2 K-Means workflow to build cross-task potential functions, and §3.8 reuses the Task 1 RF in its native 10-D form to drive enemy penalties. The factorial confirms that both reused-as-potential components yield statistically significant convergence speedups (Welch $p < 0.03$ for every cell), with the BFS oracle $>$ UL cluster-mean $>$ SL log-probability ordering matching how much domain information each Φ encodes. All three Φ choices preserve the optimal greedy trajectory empirically — the same Ng-Harada-Russell (1999) Theorem 1 result, verified across three different potential families. The §3.8 enemy factorial then closes the loop in the opposite direction: putting the Task 1 RF inside the reward function reveals the empirical Q-Learning / SARSA divergence that the uniform-step-cost baseline masks.

References

- [1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [2] Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How many random seeds? statistical power analysis in deep reinforcement learning experiments. *arXiv preprint arXiv:1806.08295*, 2018.
- [3] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In

-
- IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 886–893, 2005.
- [4] Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.
 - [5] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.
 - [6] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023.
 - [7] Tilman Lange, Volker Roth, Mikio L. Braun, and Joachim M. Buhmann. Stability-based validation of clustering solutions. *Neural Computation*, 16(6):1299–1323, 2004.
 - [8] Lorelei Lingard, Mathieu Albert, and Wendy Levinson. Grounded theory, mixed methods, and action research. *BMJ*, 337:a567, 2008.
 - [9] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
 - [10] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
 - [11] Robert R. Sokal and F. James Rohlf. The comparison of dendrograms by objective methods. *Taxon*, 11(2):33–40, 1962.
 - [12] Alexander Strehl and Joydeep Ghosh. Cluster ensembles — a knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research*, 3:583–617, 2002.
 - [13] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018.
 - [14] Robert Tibshirani and Guenther Walther. Cluster validation by prediction strength. *Journal of Computational and Graphical Statistics*, 14(3):511–528, 2005.
 - [15] Robert Tibshirani, Guenther Walther, and Trevor Hastie. Estimating the number of clusters in a data set via the gap statistic. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 63(2):411–423, 2001.
 - [16] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3-4):279–292, 1992.